UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

# Fundamentos del Deep Learning y Redes Neuronales

Semana 9 — Inteligencia Artificial


DOCENTES DEL CURSO:

**Prof. Hugo Vega / Dr. David Mauricio**

ESTUDIANTE EXPOSITOR:

Gonzalo Manuel Calero Falcon


PERIODO ACADÉMICO:

**2026-II · Ciudad Universitaria, Lima**




**Entorno académico:** Laboratorio de Cómputo e Inteligencia Artificial · Facultad de Ingeniería de Sistemas e Informática (UNMSM).

# Contenido y hoja de ruta

1. **¿Qué es el Deep Learning y cómo aprende por capas?**
   Abstracción progresiva por capas ocultas sin diseño manual.

2. **Diferencias frente al Machine Learning clásico**
   Feature engineering manual vs. representaciones automáticas de datos.

3. **Aplicaciones actuales y el problema de la caja negra**
   Diagnóstico médico, deserción estudiantil y sobreajuste.

4. **La neurona artificial y el Perceptrón Multicapa (MLP)**
   Modelo de Rosenblatt, fronteras curvas y función ReLU.

5. **Cómo aprende la red: Backpropagation y curvas de error en Python**
   Regla de la cadena, optimizador Adam y parada temprana.

6. **Caso de estudio: Predicción de deserción universitaria en la UNMSM**
   Mauricio et al., 2025 y explicabilidad SHAP.

# ¿Qué es el Deep Learning?

**El Deep Learning (o aprendizaje profundo) es una rama del Machine Learning que busca imitar la forma en que nuestro cerebro procesa la información por etapas. Cuando vemos un rostro o escuchamos una voz, no aplicamos una fórmula matemática directa, sino que nuestro cerebro reconoce primero detalles muy básicos y luego los va uniendo hasta entender el concepto completo (Goodfellow, Bengio & Courville, 2016).**

**En una computadora, este proceso se logra conectando varias capas de neuronas artificiales una detrás de otra. La palabra "profundo" no significa que el tema sea más complicado, sino simplemente que la red tiene muchas capas sucesivas trabajando en equipo para transformar datos crudos en respuestas precisas (Chollet, 2021).**




**Aplicación representativa:** Detección jerárquica de patrones y rasgos visuales en sistemas de visión artificial.

# Cómo aprende una red paso a paso

**Para entender cómo trabajan esas capas en fila, imaginemos que le damos una fotografía a la red neuronal. La primera capa oculta tiene una tarea muy sencilla: solo mira los píxeles para detectar bordes rectos, líneas inclinadas y cambios de color básicos.**

**Una vez que la primera capa encontró esos trazos, se los pasa a la segunda capa, que los junta para formar esquinas, círculos y texturas. Finalmente, la tercera capa une todas esas piezas intermedias para reconocer el objeto completo, como un automóvil o una hoja de papa enferma, sin que ningún programador haya tenido que escribir reglas manuales.**

Flujo de abstracción jerárquica por capas

Entrada · Píxeles crudos
**Matriz de intensidad luminosa y valores RGB directos sin procesar.**

Capa Oculta 1 · Rasgos primitivos
**Detección de bordes contrastados, gradientes direccionales y cambios de color.**

Capa Oculta 2 · Motivos y texturas
**Agrupación de aristas en figuras geométricas, esquinas y microestructuras.**

Capa de Salida · Inferencia semántica
**Identificación final del objeto completo.**

# ¿Por qué el Deep Learning despegó recién ahora?

**Aunque las primeras neuronas artificiales se propusieron en los años cuarenta y cincuenta, durante décadas fueron poco prácticas porque las computadoras eran lentas y casi no había datos digitales. Una red profunda necesita ver miles de ejemplos para ajustar sus conexiones sin equivocarse.**

**El gran salto ocurrió en los últimos quince años gracias a dos factores: la llegada de internet y los sensores, que generaron millones de fotos y registros (Big Data), y el uso de las tarjetas gráficas (GPU). Las GPUs permiten multiplicar miles de matrices al mismo tiempo, reduciendo entrenamientos que antes tardaban meses a solo un par de horas.**

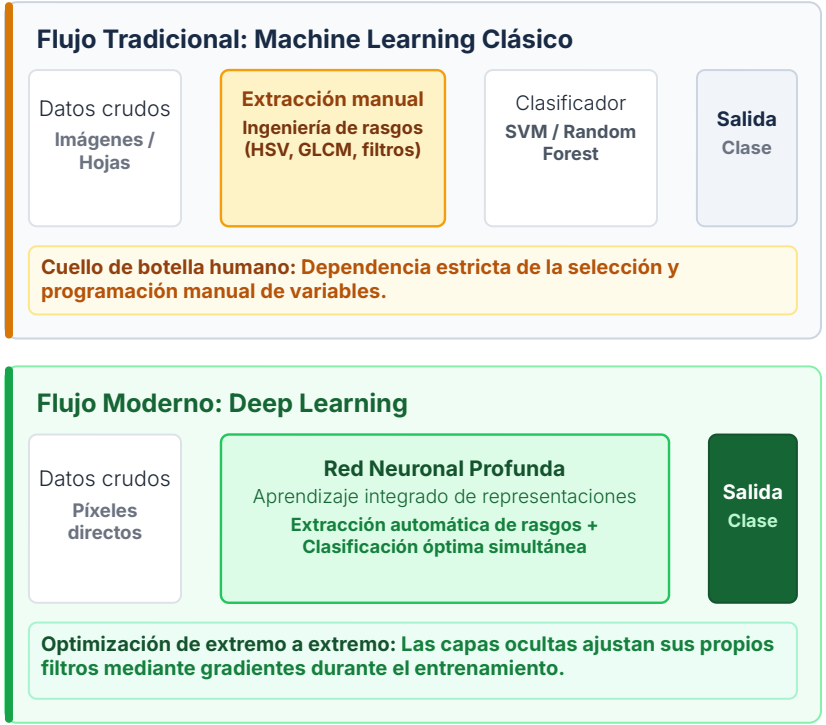



**Contexto:** Servidor de cómputo de alto rendimiento con tarjetas aceleradoras GPU para cálculo matricial paralelo masivo.

# Diferencias con el Machine Learning clásico

**En las semanas anteriores vimos modelos de** Machine Learning clásico **como SVM o Random Forest. Para que esos modelos funcionen con imágenes, el ingeniero tiene que extraer las características a mano antes de entrenar; por ejemplo, en la Semana 7 tuvimos que programar fórmulas de color** HSV **y textura** GLCM **para describir las hojas de papa.**

**En cambio, en el** Deep Learning **ya no diseñamos esas características de forma manual. Le entregamos los datos crudos directamente a la red neuronal y son sus propias capas ocultas las que descubren y ajustan automáticamente cuáles son los mejores patrones para clasificar con éxito** ***(Russell & Norvig, 2021)*****.**

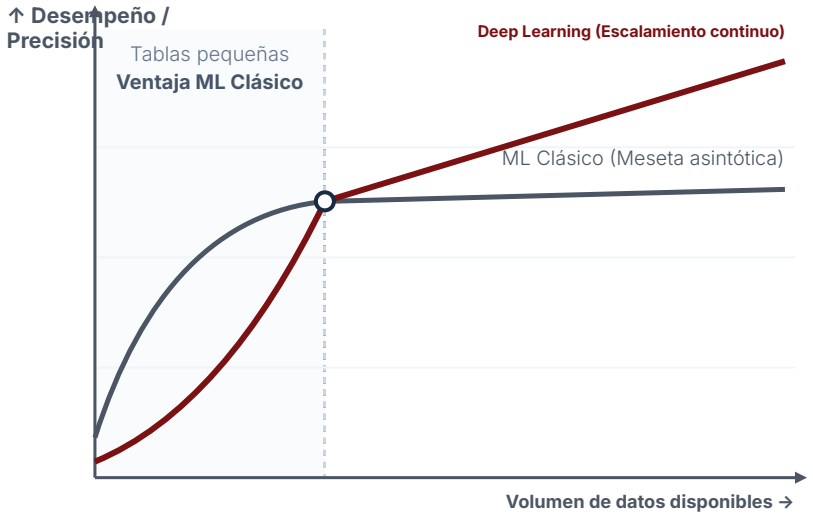


# ¿Cuándo conviene usar cada enfoque?

Datos tabulares y recursos moderados: **Un error común es pensar que el Deep Learning siempre es mejor que los algoritmos clásicos. Cuando trabajamos con tablas de datos pequeñas o medianas (por ejemplo, unos cientos o pocos miles de registros), modelos como Random Forest o SVM entrenan en segundos en cualquier laptop, no necesitan tarjeta de video y suelen dar resultados igual de buenos o mejores.**

Escalabilidad ante grandes volúmenes: **La verdadera ventaja del Deep Learning aparece cuando el volumen de datos es masivo o trabajamos con fotos, audio y texto. Mientras que los métodos clásicos llegan a un tope y dejan de mejorar aunque les demos más datos, las redes neuronales profundas siguen aumentando su precisión a medida que crece el dataset.**

Rendimiento según volumen de datos disponibles




**Conclusión analítica:** Los algoritmos tradicionales saturan su capacidad representacional, mientras que las redes neuronales profundas continúan generalizando patrones ante datos masivos y no estructurados.

# Aplicaciones actuales y sus limitaciones

**Hoy en día las** redes neuronales profundas **se utilizan en hospitales para detectar tumores y enfermedades cerebrales en tomografías** ***(Zuloaga-Rotta & Mauricio, 2024)***, **en universidades para identificar a tiempo qué estudiantes están en riesgo de dejar su carrera** ***(Quimiz-Moreira & Mauricio, 2025)***, **y en bancos para bloquear compras fraudulentas en milisegundos.**

**Sin embargo, tienen dos limitaciones que siempre debemos cuidar. La primera es que funcionan como una** "caja negra"**: aciertan con alta precisión, pero no explican por sí solas el motivo de su respuesta. La segunda es el riesgo de** sobreajuste (overfitting)**, que ocurre cuando la red memoriza los datos de entrenamiento en vez de aprender a generalizar.**



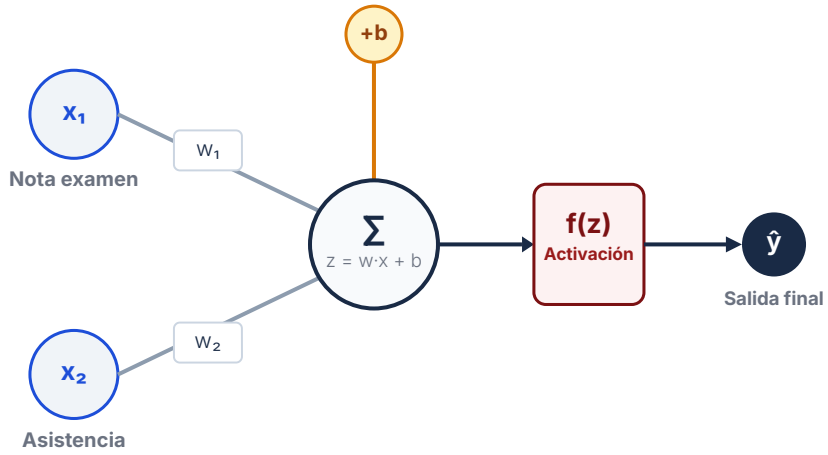
**Contexto clínico:** Detección asistida de anomalías y patologías cerebrales mediante redes neuronales en tomografías.

# La neurona artificial por dentro

**Así como el cerebro está formado por neuronas biológicas que reciben señales por sus dendritas y disparan un impulso por el axón, una red artificial está compuesta por unidades simples llamadas** perceptrones (Rosenblatt, 1958)**. Imaginemos una neurona que quiere predecir si un alumno aprobará un curso usando dos datos: su** nota de examen (x1) **y su** asistencia (x2)**.**

**La neurona multiplica cada dato por un** peso (w) **según su importancia, suma todo y le añade un ajuste base llamado** sesgo (b)**:** z = (w1 · x1 + w2 · x2) + b**. Finalmente, ese resultado** z **pasa por una función de activación que decide qué valor final entregará la neurona hacia la siguiente capa.**

Arquitectura matemática de la neurona artificial




**Ecuación matemática:** $\hat{y} = f\left( w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b \right)$

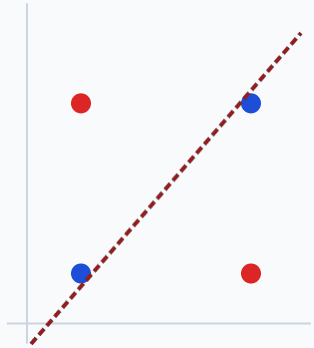
# El límite de una neurona y el Perceptrón Multicapa (MLP)

**En 1969, Marvin Minsky y Seymour Papert demostraron que una sola neurona artificial tiene una limitación importante: matemáticamente solo puede trazar una línea recta para separar dos grupos de datos. Por eso era incapaz de resolver problemas donde los casos están cruzados, como la compuerta lógica XOR o fenómenos reales donde varias variables interactúan entre sí.**

**La solución fue unir varias neuronas en capas intermedias llamadas capas ocultas, dando origen al Perceptrón Multicapa (MLP). Cuando varias neuronas combinan sus cálculos en capas sucesivas, logran doblar el espacio y trazar fronteras curvas que pueden separar cualquier patrón complejo (Teorema de Aproximación Universal).**

**Plano A: Neurona simple**

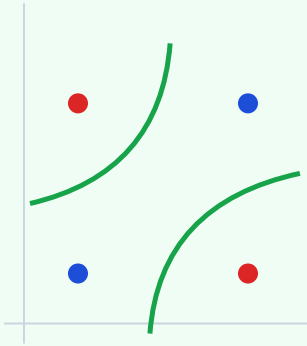
Límite lineal (1 capa)



Límite Minsky & Papert: **Hiperplano recto insuficiente para separar la distribución del problema XOR.**

**Plano B: MLP con capas ocultas**

Fronteras no lineales



Teorema Universal: **Frontera continua y envolvente no lineal que separa con éxito los patrones complejos.**

**La combinación de neuronas y funciones de activación no lineales permite aproximar cualquier frontera de decisión en el espacio de características.**

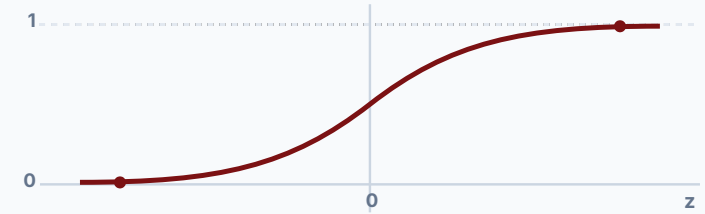
# Las funciones de activación

**Para que las capas ocultas puedan trazar esas fronteras curvas, cada neurona necesita una función de activación no lineal al final de su suma. Antiguamente se usaba mucho la función Sigmoide, que comprime cualquier número entre 0 y 1; hoy la Sigmoide se deja solo para la última capa cuando queremos obtener una probabilidad binaria (sí o no), porque en las capas internas frena el aprendizaje.**

**En las capas ocultas modernas la función estándar es ReLU (Rectified Linear Unit). Su regla es sumamente sencilla: si el número que entra es negativo lo convierte en cero, y si es positivo lo deja pasar igual. Gracias a que no aplasta los valores positivos, permite entrenar redes de muchas capas de forma rápida y estable.**

**Función Sigmoide: $\sigma(z) = 1 / (1 + e^{-z})$**

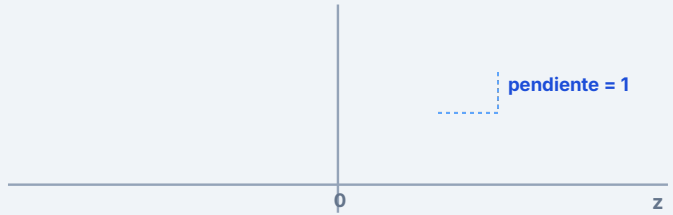
Acotada en [0, 1]




Curva en forma de 'S' acotada en [0, 1] con saturación en los extremos.

**Función ReLU: $f(z) = \max(0, z)$**

Estándar en capas ocultas



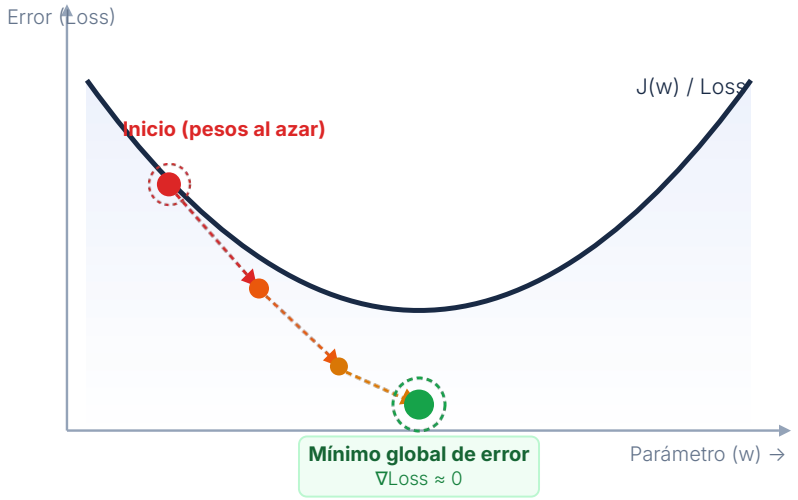

**Vale 0 para $z < 0$ y mantiene pendiente unitaria para $z \geq 0$ sin aplastar valores.**

# ¿Cómo aprende la red? El algoritmo Backpropagation

**Al crear una red neuronal, sus pesos empiezan con números al azar, por lo que sus primeras respuestas son malas. El aprendizaje consiste en corregir esos pesos vuelta tras vuelta mediante el algoritmo** Backpropagation ***(Rumelhart, Hinton & Williams, 1986)*****. Primero los datos viajan hacia adelante para lanzar una predicción y se mide cuánto se equivocó la red usando una función de pérdida (**Loss**).**

**Luego viene el paso clave: el error viaja desde la salida hacia atrás capa por capa usando derivadas (**Regla de la Cadena**) para descubrir qué peso tuvo más culpa del fallo. Con esa información, el optimizador (como** Adam**) ajusta un poco cada peso mediante el descenso del gradiente para que en la siguiente época el error sea menor.**

Superficie de coste y descenso del gradiente




Ecuación de actualización de pesos

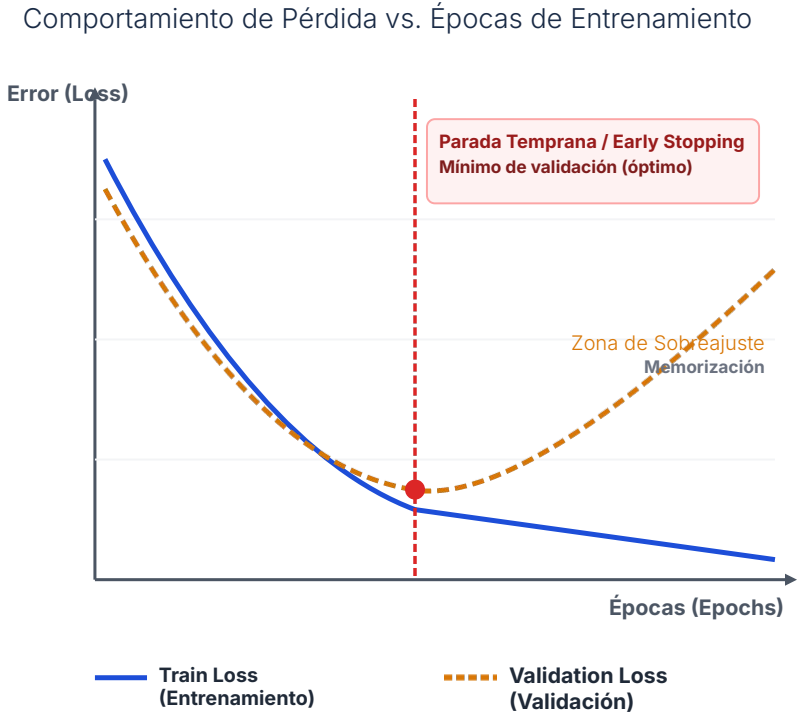
$$w_{nuevo} = w_{actual} - \eta \cdot (\partial Loss / \partial w)$$

**El gradiente negativo orienta la dirección de máxima reducción del error.**

# Cómo leer las curvas de convergencia del error

**Cuando entrenamos la red en Python, vigilamos su progreso graficando dos líneas época por época: el error en los datos de entrenamiento (**Train Loss**) y el error en datos nuevos de validación (**Validation Loss**). Al principio ambas curvas bajan juntas, lo que nos confirma que la red está comprendiendo los patrones reales del problema.**

**Sin embargo, si entrenamos durante demasiadas épocas, veremos que el error de entrenamiento sigue bajando hacia cero mientras el error de validación da la vuelta y empieza a subir. Ese momento exacto nos avisa que la red empezó a memorizar (**sobreajuste**). Para evitarlo usamos** Parada Temprana (Early Stopping)**, deteniendo el entrenamiento justo cuando la validación tocó su punto más bajo.**



# Entrenamiento de una red neuronal en Python

entrenamiento_mlp_scikit_learn.py

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

# 1. Escalar variables para que tengan el mismo rango
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# 2. Definir red con 2 capas ocultas (64 y 32 neuronas)
mlp = MLPClassifier(
hidden_layer_sizes=(64, 32),
activation='relu',
solver='adam',
alpha=0.01,
early_stopping=True,
random_state=42
)
mlp.fit(X_train_sc, y_train)
```

**Antes de pasar los datos a la red neuronal, es indispensable normalizarlos con** StandardScaler**. Si una variable tiene números grandes (como el ingreso familiar en soles) y otra tiene números pequeños (como el promedio de notas de 0 a 20), la red se desestabiliza; el escalado deja todas las columnas en condiciones iguales.**

**Al configurar** MLPClassifier**, indicamos dos capas ocultas de 64 y 32 neuronas con activación** ReLU **y el optimizador** Adam**. Además, activamos** early_stopping=True **para que Python separe automáticamente una fracción de validación y detenga el ajuste en la mejor época.**

# Caso de estudio: Deserción universitaria en la UNMSM (1/5)

**Para ver cómo se aplica una red neuronal a un problema real de nuestro entorno, analizamos la investigación publicada por el equipo del** Dr. David Mauricio **en la UNMSM: *Factors, Prediction, Explainability, and Simulating University Dropout Through Machine Learning* (Quimiz-Moreira et al., 2025), en la revista *Computation*.**

**En el Perú y América Latina, cerca del** 25% de los jóvenes **que ingresan a la universidad abandonan sus estudios durante los primeros ciclos. Los métodos estadísticos antiguos intentaban predecir este problema mirando solo el promedio de notas de forma lineal, sin tomar en cuenta cómo se cruzan las notas con los problemas económicos y familiares del estudiante.**



**Contexto:** Sesión tutorial y acompañamiento académico personalizado en la facultad.

# Las variables que alimentan la red neuronal (2/5)

**Un estudiante rara vez deja su carrera por un único motivo; casi siempre ocurre una combinación de factores que se acumulan. Por ello, los investigadores recopilaron 42 variables por alumno que ingresan directamente a las 42 neuronas de la capa de entrada de la red neuronal.**

**Estas variables abarcan cuatro aspectos de la vida del estudiante: su rendimiento académico (notas del primer ciclo, cursos desaprobados y asistencia), su situación económica (ingreso familiar y horas de trabajo), su estado personal (edad, estrés y vocación por la carrera) y el soporte que recibe de la universidad (acceso a comedor, becas y tutoría).**

### 1. Rendimiento Académico

- Promedio ponderado primer ciclo
- Cursos desaprobados
- Asistencia

### 2. Situación Económica

- Nivel de ingreso familiar mensual
- Horas de trabajo a la semana

### 3. Factores Personales

- Rango de edad
- Estrés percibido
- Orientación vocacional

### 4. Soporte Institucional

- Acceso a comedor universitario
- Becas
- Tutorías

# Preparación de los datos y entrenamiento (3/5)

**Al entrenar la red con datos históricos de matrícula, el primer reto fue que la mayoría de alumnos sí continúa sus estudios (alrededor del 75%), mientras que los que abandonan son el grupo menor (25%). Si entrenamos así de frente, la red tiende a ignorar a la minoría; por eso aplicaron primero la técnica SMOTE para equilibrar ambos grupos.**

**Luego escalaron las 42 variables y entrenaron un Perceptrón Multicapa (MLP) evaluado con validación cruzada de 10 pliegues. Gracias a sus capas ocultas con activación ReLU, la red aprendió a detectar combinaciones de alto riesgo, como el caso de un alumno que trabaja jornada completa, vive lejos del campus y reprueba un curso básico en su primer semestre.**

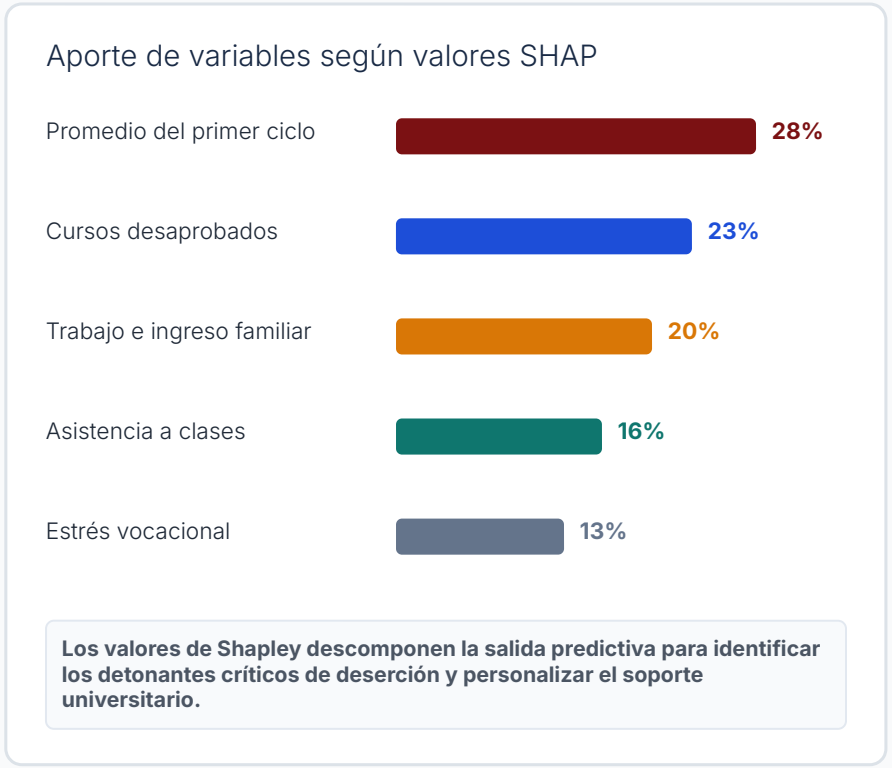



Validación experimental: **Análisis de hiperparámetros, balanceo de clases y curvas de pérdida en estaciones de trabajo del laboratorio de sistemas.**

# Resultados y apertura de la caja negra con SHAP (4/5)

**Los modelos basados en redes neuronales alcanzaron una exactitud de entre el 89% y el 94%, logrando detectar a más de 9 de cada 10 estudiantes que realmente estaban por abandonar antes de que termine el semestre. Sin embargo, entregar solo un porcentaje de riesgo no basta para que la oficina de bienestar sepa cómo ayudar al alumno.**

**Para resolver el problema de la caja negra, los autores integraron la técnica SHAP a la salida de la red neuronal. SHAP calcula cuánto aportó cada una de las 42 variables al riesgo individual de un estudiante específico, mostrándole al tutor si el peligro principal viene por el lado económico, por la carga laboral o por dificultades académicas.**

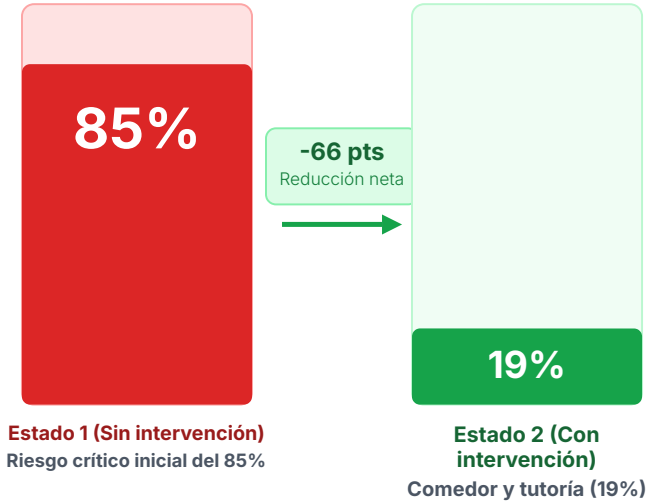


# Simulación de ayuda preventiva al estudiante (5/5)

**El aporte más práctico de la investigación de** Quimiz-Moreira y Mauricio (2025) **es que usa la red neuronal entrenada para simular soluciones antes de que el alumno deje la universidad. Cuando el sistema detecta a un estudiante con un** 85% de probabilidad de abandono**, el tutor puede probar distintos escenarios de apoyo en la computadora.**

**Por ejemplo, al simular en el modelo qué pasaría si a ese estudiante se le asigna una** beca de comedor universitario **y** acompañamiento de tutoría **en sus cursos críticos, la red recalcula su proyección y muestra que el riesgo de deserción** baja del 85% al 19%**. Así, el presupuesto de bienestar se dirige a quienes más lo necesitan.**

Simulación contrafáctica de intervención preventiva




**Reducción neta de 66 puntos porcentuales: focalización eficiente del bienestar universitario.**

# Otro caso de estudio: Diagnóstico de enfermedades cerebrales

**Esta misma filosofía de unir redes neuronales profundas con explicabilidad se aplicó también en medicina en otro estudio de nuestra facultad:** Explainable Machine Learning Models for Brain Diseases **(Zuloaga-Rotta, Salas-Ojeda & Mauricio, 2024), publicado en la revista *Neurology International*.**

**En el diagnóstico temprano de Alzheimer, Parkinson y accidentes cerebrovasculares, una red neuronal puede alcanzar** más del 90% de acierto **analizando datos clínicos e imágenes. Sin embargo, los autores demuestran que un médico neurólogo solo puede incorporar la IA en su consulta cuando el sistema incluye gráficos de explicabilidad que respalden qué indicadores médicos motivaron el diagnóstico.**

*Referencia: Zuloaga-Rotta, Salas-Ojeda & Mauricio (2024). Neurology International · UNMSM.*



**Contexto clínico:** Médicos neurólogos validando escáneres cerebrales y mapas de calor explicativos en monitor de diagnóstico.

# Ética y la Ley Peruana de IA N.° 31814

**El uso de redes neuronales en áreas sensibles como la educación y la salud debe cumplir con la Ley N.° 31814, norma que promueve el desarrollo de la Inteligencia Artificial en el Perú bajo principios éticos y de protección de los derechos fundamentales de los ciudadanos.**

**El pilar central de esta ley para los ingenieros de sistemas es la supervisión humana (Human-in-the-Loop). Ninguna red neuronal debe tomar por su cuenta una decisión definitiva sobre el futuro de una persona; su función es entregar un reporte claro y explicable para que el profesional responsable tome una decisión mejor informada.**



**Contexto:** Validación ética colegiada y supervisión humana activa frente a auditorías de algoritmos en entornos de alto impacto.

# Preguntas de repaso y discusión

### 1. ¿Por qué se prefiere ReLU frente a Sigmoide en capas ocultas?

Porque la función Sigmoide satura en los extremos y contrae los gradientes a casi cero frenando el aprendizaje en redes profundas, mientras que ReLU transmite valores positivos con derivada constante sin debilitar la señal del error.

### 2. ¿Qué ocurre si inicializamos todos los pesos w exactamente en cero?

Se produce una simetría total; todas las neuronas de una misma capa calcularían los mismos valores intermedios y recibirían exactamente las mismas actualizaciones de gradiente, impidiendo que aprendan patrones distintos.

### 3. ¿Cómo se evidencia el sobreajuste al ver la gráfica de error?

Cuando la curva de pérdida de entrenamiento continúa descendiendo hacia cero pero la curva de validación se detiene, cambia de trayectoria y empieza a ascender progresivamente.

### 4. ¿Para qué conectamos SHAP a una red neuronal?

Para abrir la condición de caja negra, asignando un valor explicativo concreto al aporte individual de cada variable de entrada respecto a la predicción emitida para un caso determinado.

# Conclusiones

## Síntesis de aprendizajes clave

**1.** El Deep Learning aprende representaciones por niveles de forma automática a partir de los datos crudos, superando el límite de diseñar variables a mano cuando contamos con grandes volúmenes de información.

**2.** Mientras una neurona aislada solo puede trazar fronteras rectas, conectar varias neuronas en capas ocultas (MLP) con funciones no lineales como ReLU permite resolver problemas complejos del mundo real.

**3.** El entrenamiento mediante Backpropagation y descenso del gradiente requiere vigilar siempre las curvas de pérdida y aplicar parada temprana (Early Stopping) para garantizar que el modelo generalice bien.

**4.** Las investigaciones desarrolladas en la UNMSM demuestran que combinar redes neuronales con explicabilidad SHAP permite aplicar el aprendizaje profundo con transparencia y alto impacto social en el Perú.

## Referencias bibliográficas


- Goodfellow, Bengio & Courville (2016). ***Deep Learning***. MIT Press.
- Chollet, F. (2021). ***Deep Learning with Python***. Manning.
- Russell, S. & Norvig, P. (2021). ***Artificial Intelligence: A Modern Approach***. Pearson.
- Quimiz-Moreira, Zuloaga-Rotta & Mauricio, D. (2025). ***Computation*** (UNMSM).
- Zuloaga-Rotta, Salas-Ojeda & Mauricio, D. (2024). ***Neurology International***.
- Congreso de la República del Perú. ***Ley N.° 31814 sobre IA***.


*Exposición académica para fines formativos en la UNMSM.*